03 串

- 串
- 串的模式匹配

03 串

- 串
- 串的模式匹配

串(String)的定义

- 设V是程序设计语言所采用的字符集，由字符集V上的字符所组成的任何有限序列，称为**字符串**(或简称为**串**)，记为$s = \text{“}a_0a_1\ldots a_{n-1}\text{”}\ (n \geq 0)$
  - s是串的名
  - 两个双引号之间的字符序列“$a_0a_1\ldots a_{n-1}$”是串的值
    - $a_i \in V\ (0 \leq i < n)$是字符集V上的字符
    - 串中字符的数目n称为串的长度(不包括串结束符\0)
    - 长度为0的串称为空串
- 一个串的子串是该串中的任一连续子序列，包含子串的串相应地称为主串
- 字符在序列中出现的序号为该字符在串中的位置，而子串在主串中的位置是该子串第零个字符在主串中的位置

串的逻辑结构

- 在逻辑结构方面，串与线性表极为相似，区别在于串的数据对象约束为字符集V

- 串的逻辑表示
  - 数据对象：$D = \{a_i \mid a_i \in V, i = 0, 1, \ldots, n-1, n \geq 0\}$
  - 数据关系：$R = \{<a_{j-1}, a_j> \mid a_{j-1} \in D \wedge a_j \in D, 1 \leq j \leq n-1\}$

串的基本操作

- 在基本操作方面，串与线性表差别很大
  - 线性表：大多以“单个元素”为操作对象，例如，在线性表中查找某个元素、在某个位置上插入一个元素或删除一个元素等
  - 串：通常以“串的整体”作为操作对象，例如，在串中查找某个子串、在串的某个位置上插入一个子串或删除一个子串等

串的基本操作

- 字符串数据结构需支持的基本操作
  - Assign：字符串的初始化
  - GetLen：获得字符串的长度
  - IsEmpty：判断当前字符串是否为空串
  - Empty：清空当前字符串，即使当前串为空串
  - Comp：比较两个字符串是否相等
  - Concat：将两个字符串拼接在一起
  - SubString：获得位置pos开始，长度为len的子串
  - Find：获得字符串中子串的出现位置
- 除上述基本操作外，还包括Insert、Delete、Replace等操作，相对串上的基本操作，这些操作相对比较复杂

串的存储结构

- **静态数组存储方法**：为字符串变量分配一个固定长度的存储空间，**一般用定长数组加以实现**

```
const int nMaxLen = 1024;  // 字符串的最大长度
class SString {
    public:
        ……
    private:
        int nLen;  // 字符串的长度
        char ch[nMaxLen+1];  // 字符串存储空间
};
```

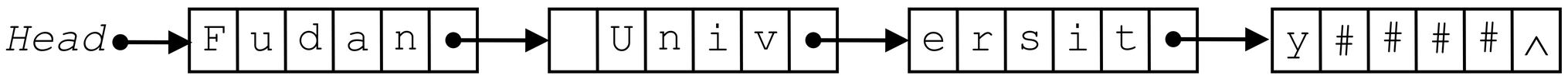
串的存储结构

- 动态数组存储方法：可以使用C++提供的new操作符分配一块连续的堆空间

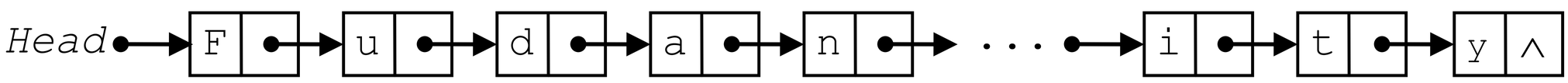
```
class DString {
    public:
        ……
    private:
        char  *ch;    // 指向当前字符串的指针
        int nLen;  // 字符串的长度
};
```

串的存储结构

- **块链存储方法**：用链表存储字符串，链表的每个结点存储长度为n的子串，即n为链表结点的大小



(a) 结点大小为 5



(b) 结点大小为 1

- 结点大小的选择和子串存储方式是影响效率的重要因素
- 串的存储开销也是另一个重要因素
  - 存储密度 = 串值所占的存储位/实际分配的存储位

串的实现

```
const int nInitLen=1024; // 初始最大长度
class DString{
   public:
      DString( ); // 构造函数：为数组分配nInitLen+1大的空间，初始为空串
      ~DString( ); // 析构函数：释放字符串所占的内存
      DString(const DString &strSrc); // 重载构造函数：用已知字符串初始化当前字符串
      DString(const char *chSrc); // 重载构造函数：用chSrc初始化当前字符串
      int GetLen( ) const { return nLen; } // 获得字符串的长度
      int IsEmpty( ) { return nLen ? 0 : 1; } // 判断当前字符串是否为空串
      void Empty( ) { nLen = 0; ch[0] = '\0'; } // 清空当前字符串
      DString GetSub(int nPos, int nCount) const; // 获得位置nPos开始，长度为nCount的子串
      char operator [ ] (int nPos) const; // 操作符重载：获得指定下标的字符
      DString & operator = (const DString &str); // 操作符重载：字符串赋值
      DString & operator += (const DString &str); // 操作符重载：字符串拼接
      int operator == (const DString &str) const; // 操作符重载：字符串等值判断
      int Find(DString &strSub) const; // 字符串的模式匹配
   private:
      int nLen; // 字符串实际长度
      char *ch; // 字符串存储所在的数组
};
```

构造与析构函数

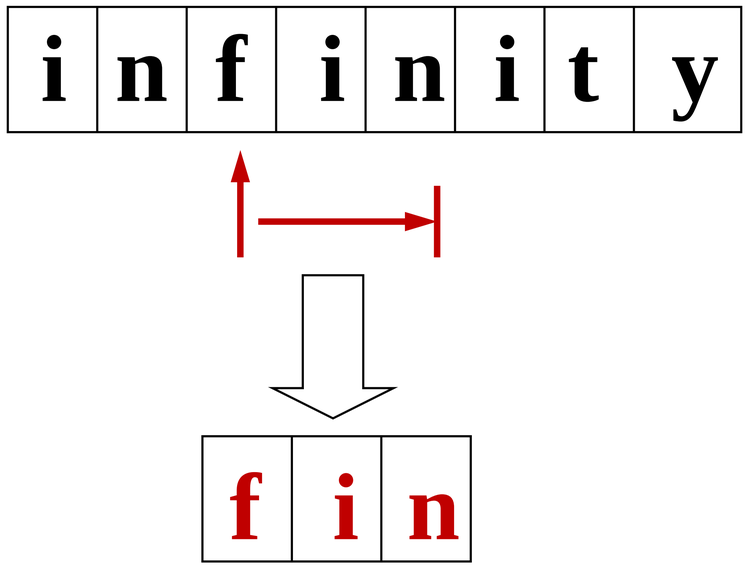
```
// 为动态数组ch开辟nInitLen+1的存储空间
DString::DString( ) {
    ch = new char[nInitLen+1];
    if ( !ch ) { cerr<<"Allocation Error!\n";  return; }
    // 初始化当前字符串：长度为0，空串
    nLen = 0;
    ch[0] = '\0';
}
// 释放动态数组的存储空间
DString::~DString( ) {
    delete [ ] ch;
}
```

构造函数重载

```
DString::DString(const DString &strSrc) {
    nLen = strSrc.GetLen( );  // 设置字符串长度
    // 为动态数组开辟max(strSrc.nLen, nInitLen)+1的存储空间
    if (nLen > nInitLen)  ch = new char[nLen+1];
    else  ch = new char[nInitLen+1];
    if ( !ch ) { cerr<<"Allocation Error!\n";  return; }
    strcpy(ch, strSrc.ch);  // 复制字符串序列
}
DString::DString(const char *chSrc) {
    nLen = strlen(chSrc);  // 设置字符串长度
    // 为动态数组开辟max(strlen(chSrc), nInitLen)+1的存储空间
    if (nLen > nInitLen)  ch = new char[nLen+1];
    else  ch = new char[nInitLen+1];
    if ( !ch ) {cerr<<"Allocation Error!\n";  return; }
    strcpy(ch, chSrc);  // 复制字符串序列
    }
```

提取子串函数

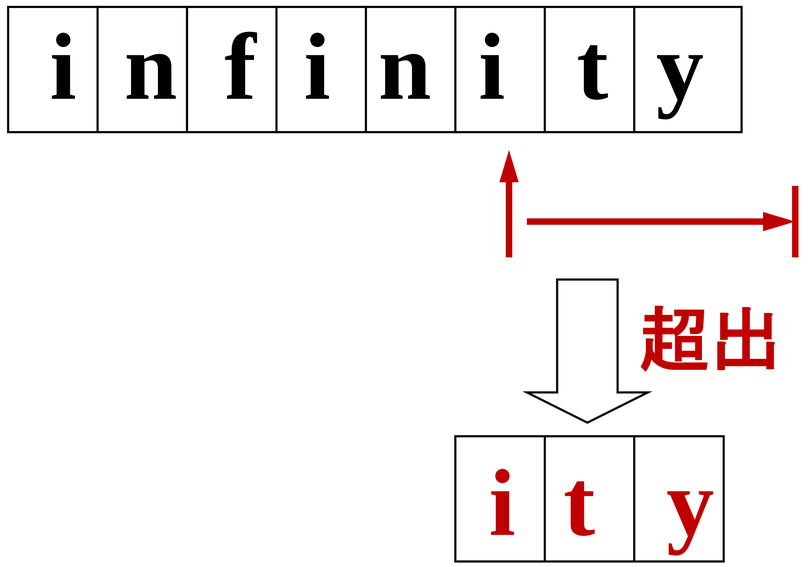
nPos=2, nCount=3



nPos+nCount-1 ≤ nLen-1

- nPos+nCount-1：待提取子串最后一个字符在串中的位置
- nLen-1：串的最大允许位置

nPos=5, nCount=4



nPos+nCount-1 ≥ nLen

提取子串函数

```
DString DString::GetSub(int nPos, int nCount) const {
    DString tmpString;  // 定义临时字符串，作为保存子串的临时对象
    char * pch;  // 字符串指针，指向要获取的子串
    if (nPos < 0 || nCount < 0) {  // 判断输入参数是否合理
        tmpString.nLen = 0;
        tmpString.ch[0] = '\0';
    } else {  // 将子串拷贝到tmpString对象中
        if ( nPos + nCount - 1 >= nLen)  //长度越界
            nCount = nLen - nPos;
        tmpString.nLen = nCount;
        pch = ch + nPos;
        memcpy(tmpString.ch, pch, nCount);
        tmpString.ch[nCount] = '\0';
    }
    return tmpString;  // 返回获取到的子串
}
```

获取字符函数

```
// 获取当前串的第i个字符
char DString::operator [ ] (int nPos) const {
    if ( nPos < 0 && nPos >= nLen) {
        cerr<<“nPos Out Of Bounds!\n";
        return 0;
    }
    return ch[nPos];
}
```

串赋值函数

```
DString & DString::operator = (const DString &str) {
    if ( &str != this) {
        delete [ ] ch;  // 释放原有的字符串存储空间
        nLen = str.nLen;  // 复制字符串的长度
        // 若新的长度大于默认的初始化长度，则按新长度申请存储空间
        if ( nLen > nInitLen )  ch = new char[nLen+1];
        else  ch = new char[nInitLen+1];  // 否则按默认长度申请存储空间
        if ( !ch ) { cerr<<"Allocation Error!\n"; exit(-1); }
        strcpy(ch, str.ch);  // 复制字符串序列
    }
    return *this;
}
```

串拼接函数

```
DString & DString::operator += (const DString &str) {
    char * temp = ch; // 暂存原串数组
    int n = nLen + str.nLen;
    int m = (nInitLen >= n) ? nInitLen : n;
    ch = new char[m+1];  // 申请新的存储空间
    if ( !ch ) { cerr<<"Allocation Error!\n";  exit(-1); }
    nLen = n;
    strcpy(ch, temp); // 复制原串数组
    strcat(ch, str.ch); // 连接str串数组
    delete [ ] temp;
    return *this;
}
```

串等值判断函数

```
// 字符串等值判断：相等则返回0
int DString::operator == (const DString &str) const {
    return strcmp(ch, str.ch);
}
```

练习1

- 设S为一个长度为n的字符串，其中的字符各不相同，求解S中互异的非平凡子串(非空且不同于S本身)的个数
- 除长度为n的子串外，长度为n-1的不同子串个数为2，长度为n-2的不同子串个数为3，依此类推，长度为1的不同子串个数为n，即S的非平凡子串个数为2 + 3 + … + n = n(n+1)/2-1

练习1

- 如果字符串的一个子串(其长度大于1)的各个字符均相同，则称之为等值子串。试设计一个算法，输入字符串S，以“!”作为结束标志，如果S中不存在等值子串，则输出“无等值子串”，否则输出一个长度最大的等值子串
- 设head指向当前发现的最长等值子串的串头，max纪录此子串的长度。遍历串S，若发现等值子串则用length记录其长度，如果length大于max，则对head和max进行更新。最后根据head和max输出结果

03 串

- 串

- **串的模式匹配**

串的模式匹配

- **串的模式匹配**解决的是在长串(称为**text**)中查找短串(称为**pattern**)的一个、多个或所有出现的问题

- 一个长度为**m**的**pattern**可被表示为**p = p[0..m-1]**，长度为**n**的**text**可被表示为**t = t[0..n-1]**，而串的模式匹配任务就是在**t**中找到**p**出现的位置(即**p**的第一个字符出现的位置)

- 模式匹配过程中，查看text中长度为m的窗口，用pattern串和text的窗口中的子串进行比对。比对完成后，将窗口向右滑动，并不断重复这一过程，直到根据需要找到所需匹配为止。这种机制被称为**滑动窗口机制**

BF(Brute-Force)算法

- 依次比对pattern和滑动窗口中对应位置上的字符，比对完成后将滑动窗口向右移动1，直到找到所需的匹配为止

t: C G T A G C G T C T C T C A T A T G T C A T G C

p: C G T C T C T C

t: C G T A G C G T C T C T C A T A T G T C A T G C

p: C G T C T C T C

t: C G T A G C G T C T C T C A T A T G T C A T G C

p: C G T C T C T C

……

t: C G T A G C G T C T C T C A T A T G T C A T G C

p: C G T C T C T C

BF算法实现

```
int DString :: Find(DString &pat) const {
    char * p = pat.ch, * t = ch;
    int i = 0;
    while ( *p != '\0' && *t != '\0')
        if ( *p++ != *t++) {  // 比较串字符，不等
            // 对齐目标的下一位置，继续比较
            i++;  t = ch + i;  p = pat.ch;
        }
    if (*p == '\0')  return i;  // 相等
    else return -1;
}
```

BF算法时间复杂度分析

- 设pattern串p有m个字符，而text串t有n个字符
- **最好情况**：text的前m个字符与pattern匹配，可在m次比较后找到匹配结果，算法复杂度为O(m)
- **最坏情况**：每次前面的字符总能匹配上，但最后的字符总匹配不上(如p="aac", t="aaaaaaaa" (m=3, n=8))，则必须比较m个字符共(n-m+1)次，即共进行m(n-m+1)次，而m(n-m+1) <= m(n-m+m) = mn，故算法复杂度为O(mn)

BF算法的特点

- 不需要预处理过程
- 只需固定的存储空间
- 滑动窗口的移动每次都为1
- 字符串的比对可按任意顺序进行(从左到右、从右到左、或特定顺序均可)
- 最大比较次数为(n-m+1)m
- 算法的时间复杂性为O(mn)

KR(Karp-Rabin)算法

- 作为最朴素的字符串匹配算法，BF算法的效率并不理想
  - 一是子串与滑动窗口内的子串逐个字符匹配所引发的效率问题
  - 二是该算法太健忘，前一次匹配的信息其实可以有部分应用到后一次匹配中，而BF算法只是简单的把这个信息扔掉，重头再来

- KR算法针对“滑动窗口内容逐一匹配”的问题，将滑动窗口内m个字符的比较变为一个哈希值的比较
  - 比较子串的哈希值与滑动窗口内子串的哈希值，仅当这两个哈希值相等时，再比较子串与滑动窗口内的子串是否相等

KR算法框架

```
function RabinKarp(string s[0..n-1], string sub[0..m-1])
    hsub=hash(sub[0..m-1]);  // 计算子串的哈希值
    for i from 0 to n-m  // 依次比较窗口内子串与给定子串是否相同
        hs = hash(sub[i..i+m-1]);  // 计算窗口内子串的哈希值
        if hs = hsub
            if s[i..i+m-1] = sub
                return i;  // 如果相同则记录位置并退出函数
return not found;  // 没有发现，记录没有发现给定子串
```

哈希算法

- KR算法的效率取决于哈希函数的的选取，通常，KR算法使用$h(x) = x \bmod q$作为哈希函数

- 滑动窗口内子串的哈希值计算公式：$hash(w[0..m-1]) = (w[0]\times2^{m-1} + w[1]\times2^{m-2} + \ldots + w[m-1]\times2^{0}) \bmod q$，其中，q为一个大整数

- 当滑动窗口右移时，需要对当前窗口内的子串重新利用哈希函数计算，其计算公式为：$rehash(a, b, h) = ((h-a\times2^{m-1})\times2+b) \bmod q$，其中，h为上一滑动窗口内子串的哈希值，a为即将移出滑动窗口的字符，b是即将移入滑动窗口的字符

KR算法实现

```
#define Rehash(a, b, h)  ((((h)-(a)*d)<<1)+(b))
int KR(char *p, int m, char *t, int n) {
    int d, hp, ht, i, j;
    for ( d = i = 1; i < m; i++ )  // 预处理，计算d=2^(m-1)
        d = d<<1;
    for ( hp = ht = i = 0; i < m; i++ ) {  // 计算t的初始窗口和p的哈希值
        hp = (hp<<1) + p[i];  ht = (ht<<1) + t[i];
    }
    for ( j = 0; j <= n-m; j++ ) {
        if ( hp == ht && memcmp(p, t+j, m) == 0 )
            return j;  // 如字符串相等，返回位置
        ht = Rehash(t[j], t[j+m], ht);  // 否则对t上的子串重新进行哈希
    }
    return -1;
}
```

最大权重

KR算法示例

t:　C G T A G C G T C T C T C A T A T G T C A T G C

p:　C G T C T C T C

Hash(t[0..7]) = 17910 != Hash(p)

t:　C G T A G C G T C T C T C A T A T G T C A T G C

p:　　C G T C T C T C

Hash(t[1..8]) = 18735 != Hash(p)

……

t:　C G T A G C G T C T C T C A T A T G T C A T G C

p:　　　　C G T C T C T C

Hash(t[5..12]) = 18055 = Hash(p)

练习2

- **KR算法时间复杂度分析**
- **预处理的时间复杂度为O(m)** //计算d
- **Rehash的时间复杂度为O(1)**
- **最坏情况：每次匹配都出现哈希冲突，而且对每个冲突都进行了m次比较，故时间复杂度为O(nm)。算上哈希算法的时间开销，其时间复杂度比BF算法还高**
- **当仔细地选择q时，最坏情况几乎不会出现，且m < n，故期望的时间复杂度为O(n)**

KR算法的特点

- 利用哈希的方法
- 预处理需要O(m)的时间和常数的存储空间
- 滑动窗口的移动每次都为1
- 最坏情况下，算法时间复杂性为O(mn)
- 算法时间复杂性的预期为O(n)

KMP(Knuth-Morris-Pratt)算法

- BF和KR算法的一个共同点是每次比较的窗口向右滑动的距离均为1，不够高效

- KMP算法的核心思想：利用pattern的内容指导滑动窗口向右滑动的距离

简单模式匹配的缺点：无谓比较

第1趟　t　a b b a b a

　　p　a b a

第2趟　t　a b b a b a

　　p　a b a

第3趟　t　a b b a b a

　　p　a b a

第4趟　t　a b b a b a

　　p　a b a

⇨ $p_0 != p_1 = t_1$

⇩

第2趟可以不比较

$p_0 = p_2 != t_2$

⇩

第3趟可以不比较

⇩

可以跳过第2和第3趟，直接执行第4趟，消除了回溯

KMP算法

$$
\begin{array}{lccccccccccc}
t & t_0 & \cdots & t_{s-1} & t_s & t_{s+1} & t_{s+2} & \cdots & t_{s+j-1} & t_{s+j} & \cdots & t_{n-1} \\
 & & & & \| & \| & \| & & \| & \times & & \\
p & & & & p_0 & p_1 & p_2 & \cdots & p_{j-1} & p_j & &
\end{array}
$$

第s+1趟比较

此时满足$p_0\ p_1\ p_2 \cdots p_{j-1} = t_s\ t_{s+1}\ t_{s+2} \cdots t_{s+j-1}$

为使p与t的下一趟能匹配，必须满足

$p_0\ p_1\ p_2 \cdots p_{j-1} \cdots p_{m-1} = t_{s+1}\ t_{s+2}\ t_{s+3} \cdots t_{s+j} \cdots t_{s+m}$

若$p_0\ p_1 \cdots p_{j-2} \neq p_1\ p_2 \cdots p_{j-1}$，则可以断定

$p_0\ p_1 \cdots p_{j-2} \neq t_{s+1}\ t_{s+2} \cdots t_{s+j-1}$，即下一趟必不匹配

KMP算法

同样，若$p_0\ p_1 \cdots p_{j-3} \neq p_2\ p_3 \cdots p_{j-1}$，则也可以断定
$p_0\ p_1 \cdots p_{j-3} \neq t_{s+2}\ t_{s+3} \cdots t_{s+j-1}$，即再下一趟也必不匹配

直到对于某一个“k”值，使得

$p_0\ p_1 \cdots p_{k+1} \neq p_{j-k-2}\ p_{j-k-1} \cdots p_{j-1}$

$p_0\ p_1 \cdots p_k = p_{j-k-1}\ p_{j-k} \cdots p_{j-1}$

则$p_0\ p_1 \cdots p_k = t_{s+j-k-1}\ t_{s+j-k} \cdots t_{s+j-1}$

$\|\quad\quad\|\quad\quad\quad\|$

$p_{j-k-1}\ p_{j-k} \cdots p_{j-1}$

KMP算法

- 在第s+1趟比较失配时的p从当时位置向右滑动j-k-1位

- 由于t中t_{s+j}之前的字符已经与p中p_{k+1}之前的字符匹配，因此可以直接从t中的t_{s+j}(即上一趟失配的位置)与p中p_{k+1}开始，继续向下进行匹配比较

- 由此可见，t在第s+1趟比较失配时，其扫描指针不必回溯，即算法下一趟继续从此处开始向下进行匹配比较；而p的扫描指针应回退到p_{k+1}位置

KMP算法

- Knuth-Morris-Pratt (KMP)算法发现，对于不同的j，k的取值仅依赖于p本身的前j个字符，而与t无关

- 1970年， S. A. Cook在进行抽象机的理论研究时证明了在最差情况下模式匹配可在n+m时间内完成

- 此后， D. E. Knuth和V. R. Pratt以此理论为基础，构造了一种方法来在n+m时间内进行模式匹配；与此同时，J. H. Morris在开发文本编辑器时为了避免在检索文本时的回溯也得到了同样的算法

k的确定方法

- 可以用失效函数next(j)来描述：当p中的第j个字符与t中相应字符失配时，p中应该由哪个字符与t中刚失配的字符对齐继续进行比较
  - next(j) = -1，　当j = 0
  - next(j) = k+1，　当$0 \le k < j-1$且k是使得$p_0\ p_1 \cdots p_k = p_{j-k-1}\ p_{j-k} \cdots p_{j-1}$成立的最大整数
  - next(j) = 0，　其它情况



j	0	1	2	3	4	5	6	7
p	a	b	a	a	b	c	a	c
next(j)	-1	0	0	1	1	2	0	1

利用失效函数next(j)的匹配过程

- 若j = 0，一开始就不匹配，则t的扫描指针加1，p的扫描指针不变(已经在p_0)
- 若j > 0，比较中途不匹配，则t的扫描指针不变，p的扫描指针回到 $p_{next(j)}$

第1趟 t a c a b a a b a a b c a c a a b c
p a b a a b c a c

第2趟 t a c a b a a b a a b c a c a a b c
p a b a a b c a c

第3趟 t a c a b a a b a a b c a c a a b c
p a b a a b c a c

第4趟 t a c a b a a b a a b c a c a a b c
p a b a a b c a c

j = 1 ⇨ next(j) = 0

j = 0 ⇨ next(j) = -1

j = 5 ⇨ next(j) = 2

带失效函数的KMP算法实现

```
int DString :: Find(DString &pat) const {
    int posP = 0, posT = 0;
    int lengthP = pat.nLen, lengthT = nLen;
    while ( posP < lengthP && posT < lengthT )
        if ( pat.ch[posP] == ch[posT] ) {
            posP++;  posT++;  // 相等继续比较
        } else if ( posP == 0 )  posT++;  // 一开始就不相等
        else posP = pat.next[posP]; //中途不相等
    if ( posP < lengthP )  return -1;
    else return posT - lengthP; //比较成功，返回子串第一个字符的位置
}
```

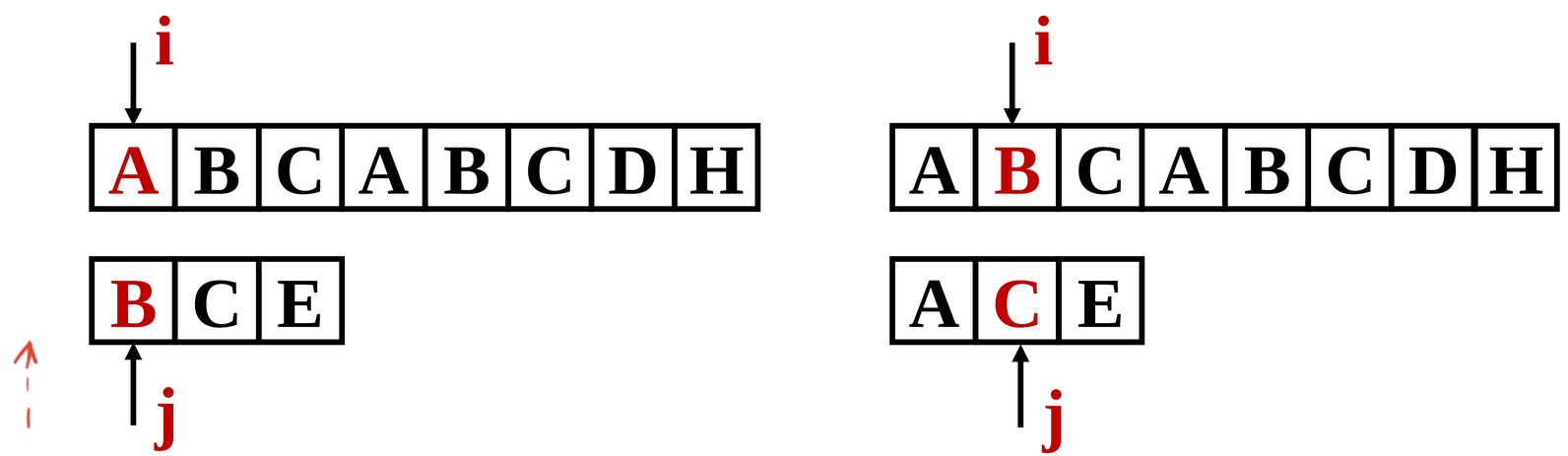
text的扫描指针在每趟中不回退，故其时间复杂度为O(n)

带失效函数的KMP算法实现

```
int DString :: Find(DString &pat) const {
    int posP = 0, posT = 0;
    int lengthP = pat.nLen, lengthT = nLen;
    while ( posP < lengthP && posT < lengthT )
       if ( posP == -1 || pat.ch[posP] == ch[posT] ) {
          posP++;  posT++;  // 相等继续比较
       } else  posP = pat.next[posP];  // 不相等
    if ( posP < lengthP)  return -1;
    else return posT - lengthP;
}
```

计算next数组

- 当j = 0时，即在第0个位置发生不匹配时，j已经在最左边，而i应该后移，故next(0) = -1

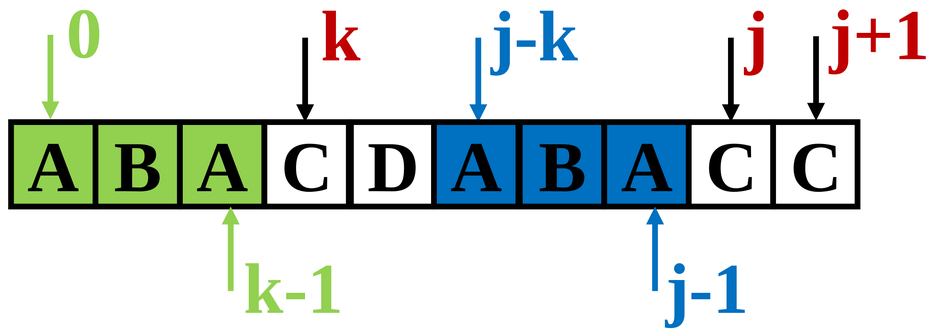


- 当j = 1时，next(1) = 0

计算next数组

- 设next(j) = k，则有$p_0\ p_1 \cdots p_{k-1} = p_{j-k}\ p_{j-k+1} \cdots p_{j-1}$

- 若$p_k = p_j$，则有$p_0\ p_1 \cdots p_{k-1}\ p_k = p_{j-k}\ p_{j-k+1} \cdots p_{j-1}\ p_j$，故 next(j+1) = k + 1 = next(j) + 1



计算next数组

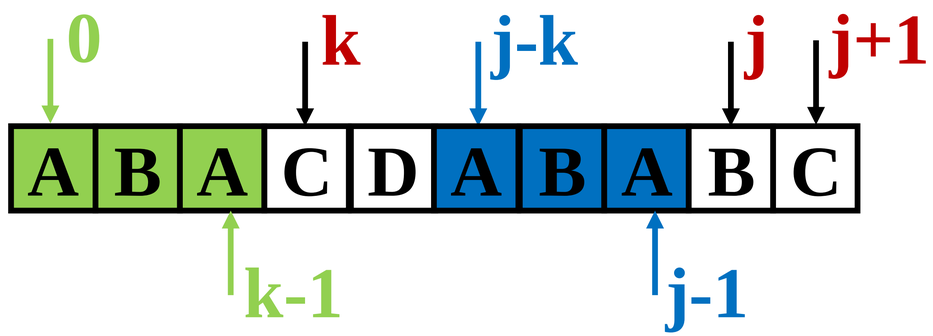
- 设next(j) = k，则有$p_0\ p_1 \cdots p_{k-1} = p_{j-k}\ p_{j-k+1} \cdots p_{j-1}$

不可能比k更长了，否则与next(j)最大矛盾

- 若p_k != p_j，则找$p_0\ p_1 \cdots p_{k-1}$中的最长前缀与$p_{j-k+1}\ p_{j-k+2} \cdots p_{j-1}\ p_j$中的后缀匹配，即找$p_0\ p_1 \cdots p_{k-1}\ p_j$中的最长前缀与其后缀匹配，而$p_0\ p_1 \cdots p_{k-1}$中的最长前缀与其后缀匹配的长度是next(k)，则需要比较$p_{next(k)}$和p_j

合并

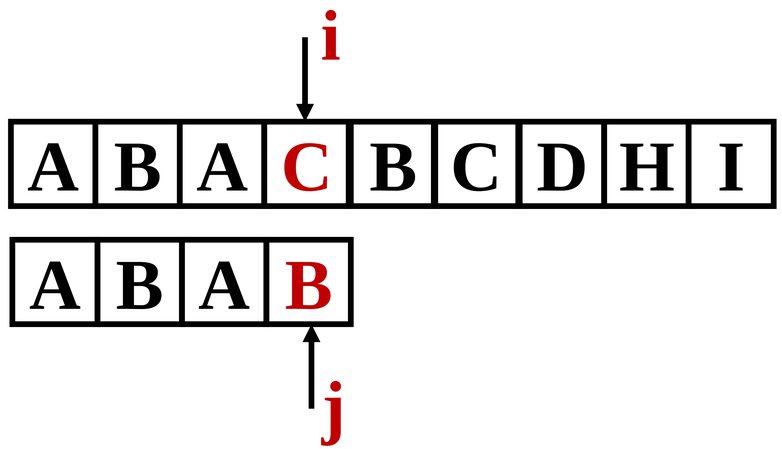
递推到前一种情况



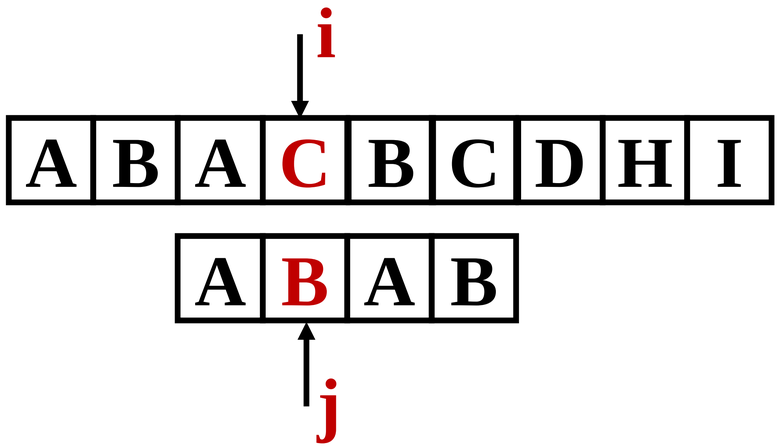
计算next数组

```
void preprocessing (char *p, int m, int next[ ]) {
    int j = 0, k = -1;
    next[0] = -1;
    while ( j < m - 1) {
        if ( k == -1 || p[j] == p[k] ) {  //防止数组越界
            j++;  k++;
            next[j] = k;   //相等，next(j+1)=k+1
        } else
            k = next[k];    //不相等，递推
    }
}
```

进一步优化next数组



前述算法得到的next数组是[-1, 0, 0, 1]，则下一步应该把j移动到第1个元素



这一步没有意义，因为后面的B已经不匹配，那前面的B也一定是不匹配的

发生问题的原因在于p[j] == p[next[j]]

k

进一步优化next数组

```
void preprocessing (char *p, int m, int next[ ]) {
    int j = 0, k = -1;
    next[0] = -1;
    while ( j < m - 1) {
        if ( k == -1 || p[j] == p[k] ) {
            j++;  k++;
            if ( p[j] == p[k] )  next[j] = next[k];
            else  next[j] = k;
        } else
            k = next[k];
    }
}
```

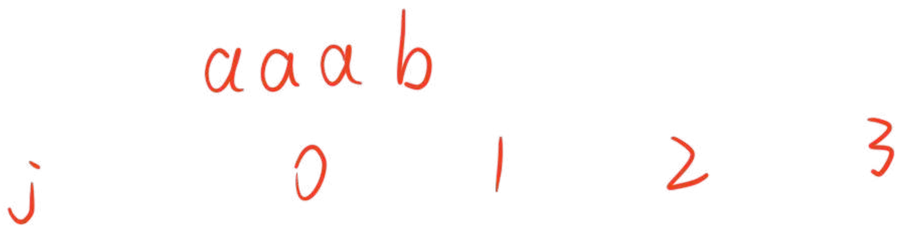
倒退两步，递推

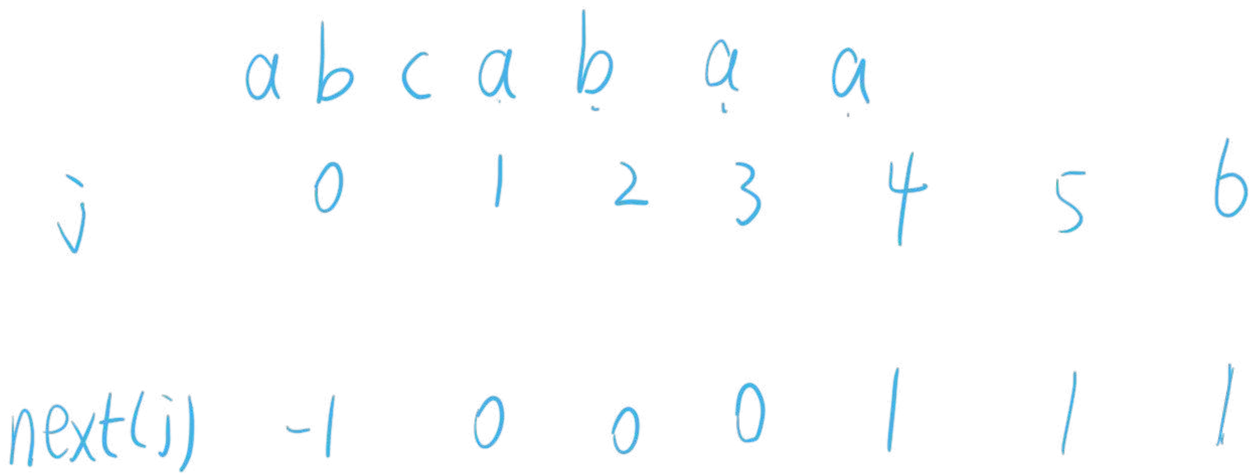
KMP算法的特点

- 与BF算法类似，按照从左到右的顺序进行比较
- 预处理需要O(m)的时间和存储空间
- 算法时间复杂性为O(m+n)

练习3

- 试计算“aaab”、“abcabaa”和“abcaabbabcabaacbacba”的失效函数





BM(Boyer-Moore)算法

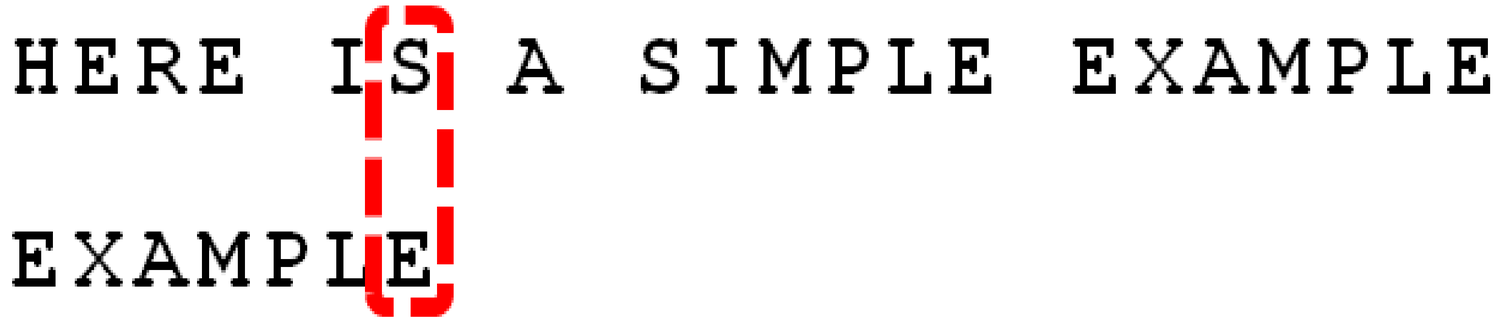
- KMP算法并不是效率最高的算法，实际采用并不多
- 各种文本编辑器的查找功能(Ctrl+F)大多采用BM算法
- BM算法不仅效率高，而且构思巧妙，容易理解
- 1977年，德克萨斯大学的Robert S. Boyer教授和J. Strother Moore教授发明了这种算法

BM算法

- BM算法与KMP算法的差别在于
  - 在进行匹配比较时，不是自左向右进行，而是自右向左进行
  - 预先计算出text中可能的字符在pattern中出现的位置相关信息，利用这些信息来减少比较次数

- 根据字符串匹配的思想，窗口向右移动的动作一定发生在匹配失效或完全匹配时。为了右移更大的距离，BM算法使用两个预计算函数来指导窗口向右移动的距离
  - 好后缀移动(Good-Suffix Shift)
  - 坏字符移动(Bad-Character Shift)

BM算法示例

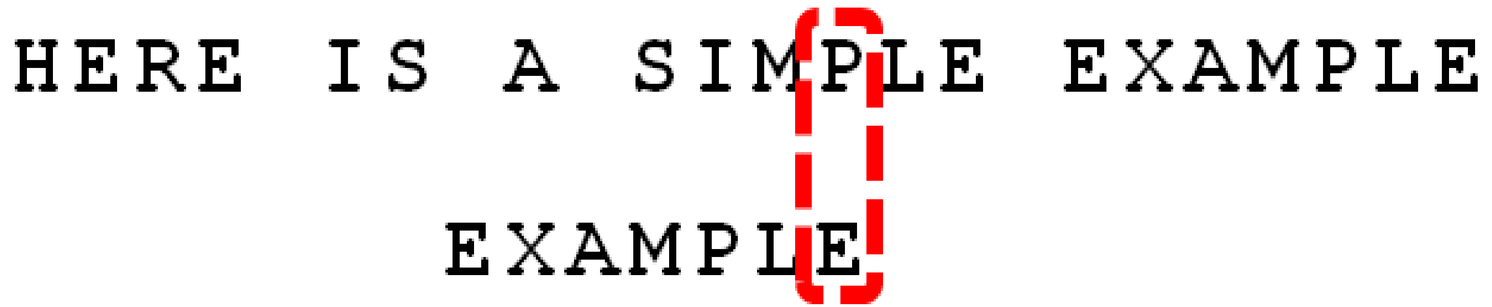


- 1) “目标串”与“模式串”头部对齐，从尾部开始比较

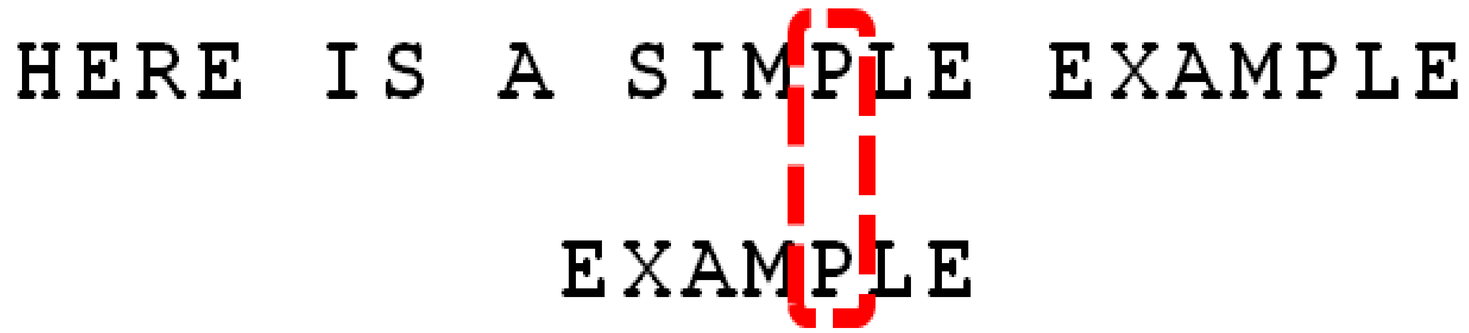
- 2) “S”与“E”不匹配。这时，“S”就被称为“坏字符”(bad character)，即不匹配的字符

- 3) “S”不包含在模式串“EXAMPLE”之中，这意味着可以把模式串直接移到“S”的后一位

BM算法示例



- 依然从尾部开始比较，发现“P”与“E”不匹配，所以“P”是“坏字符”。但是，“P”包含在模式串“EXAMPLE”之中。所以，将模式串后移两位，两个“P”对齐



BM算法示例

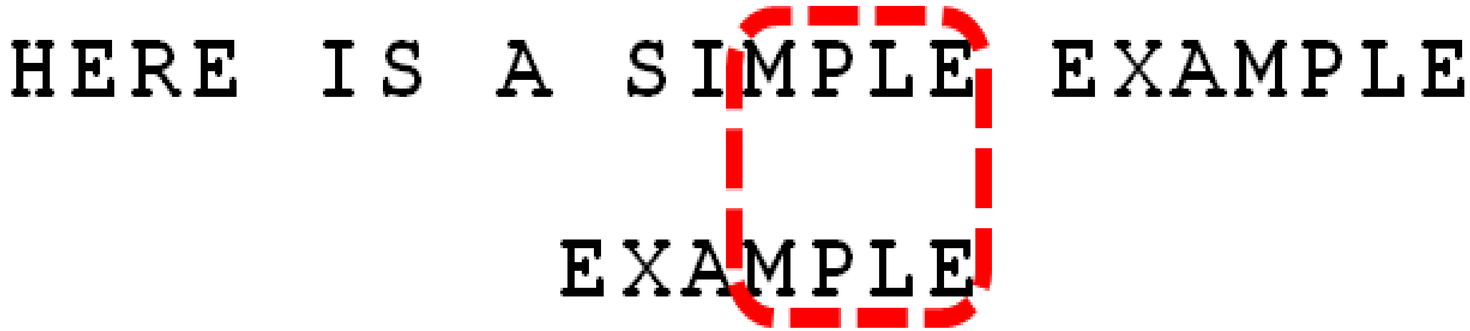
- 5) “坏字符规则”

后移位数 = 坏字符的位置 - 模式串中上一次出现位置(若“坏字符”不包含在模式串之中，则上一次出现位置为 -1)

向右移整个字符串长度

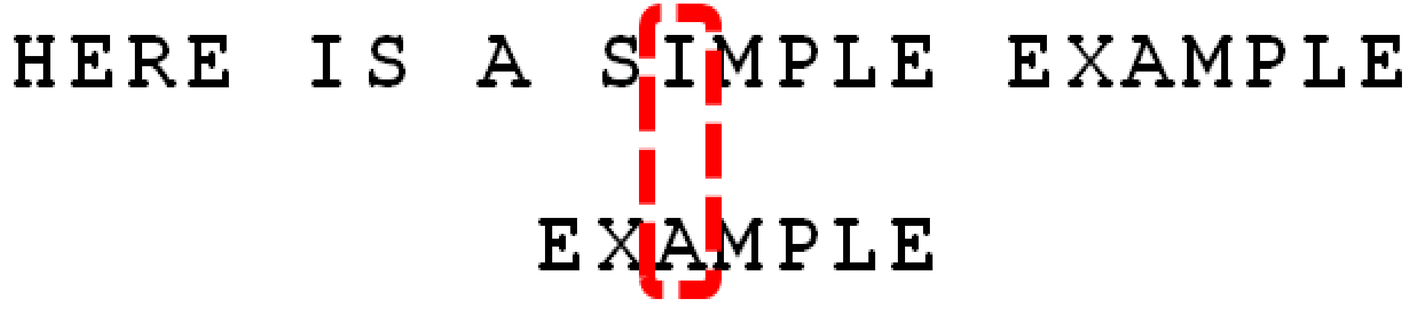
- 以“P”为例，它作为“坏字符”，出现在第6位(从0开始编号)，在模式串中上一次出现位置为4，故后移 6 - 4 = 2位
- 再以第二步的“S”为例，它出现在第6位，在模式串中上一次出现位置是-1(即未出现)，故后移 6 - (-1) = 7位

BM算法示例

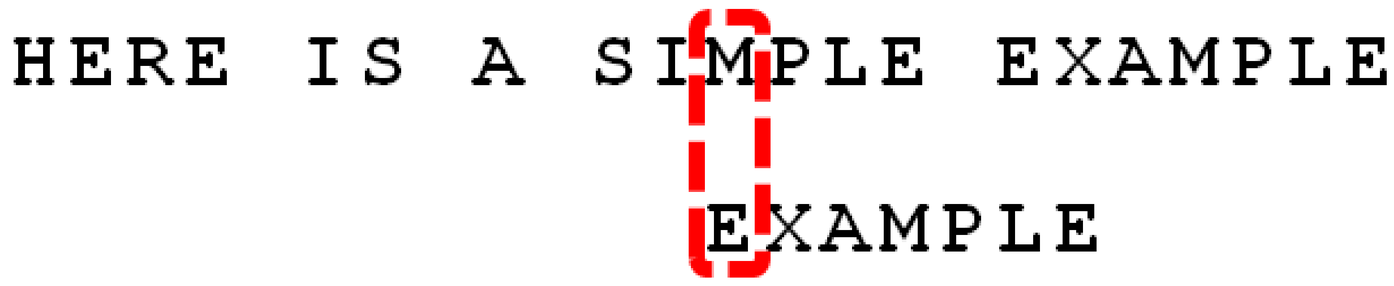


- **6) 依次从尾部比较，发现“MPLE”与“MPLE”匹配，把它称为“好后缀”(good suffix)，即所有尾部匹配的字符串，如“MPLE”、“PLE”、“LE”、“E”，都是“好后缀”**

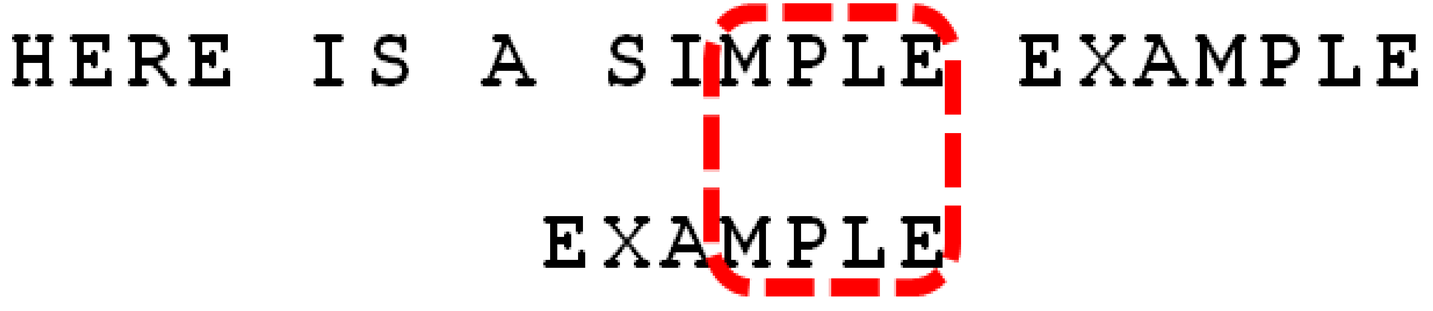
BM算法示例



- 7) 比较前一位，发现“I”与“A”不匹配，故“I”是“坏字符”
- 8) 根据“坏字符规则”，此时模式串应该后移 2 – (-1) = 3 位。此时有没有更好的移法?



BM算法示例



- **9) 此时存在“好后缀”，可采用“好后缀规则”：**

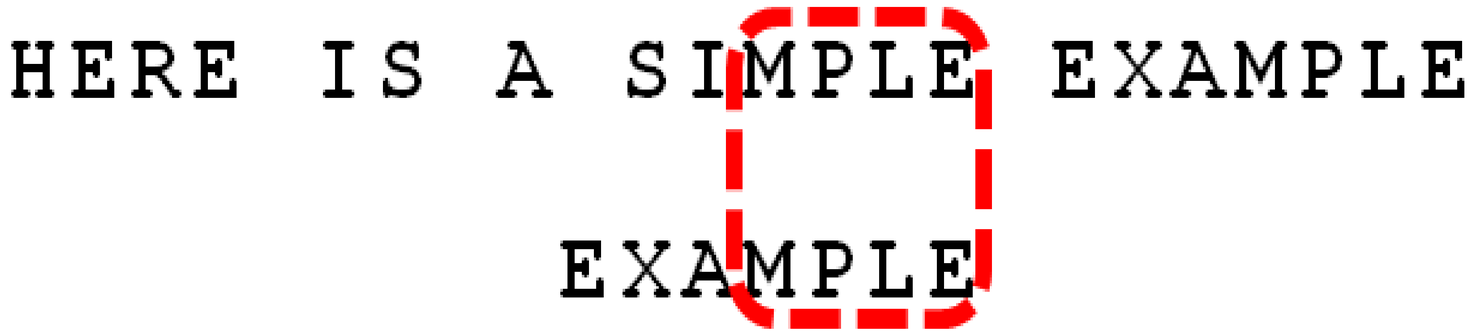
后移位数 = 好后缀的位置 - 模式串中上一次出现位置

(1) “好后缀”的位置以最后一个字符为准

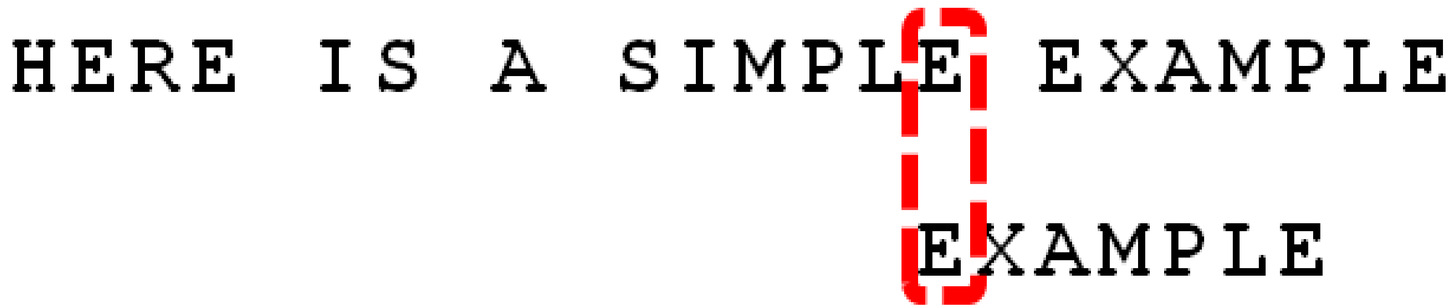
(2) 若“好后缀”在模式串中只出现一次，则它在模式串中上一次出现位置为-1

(3) 若“好后缀”有多个，则除了最长的那个“好后缀”，其它“好后缀”的上一次出现位置必须在头部

BM算法示例

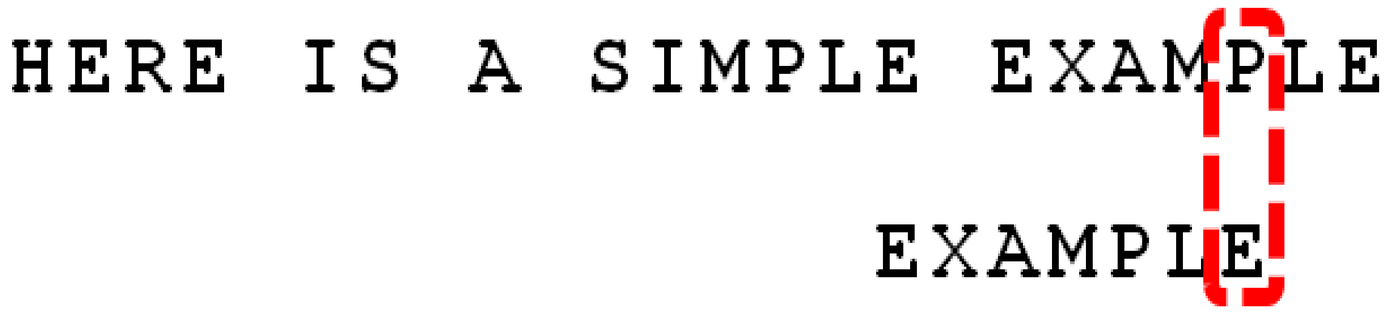


- 6) “好后缀”(“MPLE”、“PLE”、“LE”、“E”)中，只有“E”在“EXAMPLE”还出现在头部，故后移6 - 0 = 6位



“坏字符规则”移3位，“好后缀规则”移6位，故BM算法的基本思想是，每次后移这两个规则之中的较大值

BM算法示例



- 7) 继续从尾部开始比较，“P”与“E”不匹配，故“P”是“坏字符”。根据“坏字符规则”，后移6 - 4 = 2位



- 8) 从尾部开始比较，发现全部匹配，搜索结束。若还要继续查找(即找出全部匹配)，则根据“好后缀规则”，后移6 - 0 = 6位，即头部的“E”移到尾部的“E”的位置

本章小结

- 串
- 串的模式匹配

作业

- 编写一个算法frequency，统计在一个输入字符串中各个不同字符出现的频度

- 给定p=“abcabaa”与t=“abcaabbabcabaacbacba”，求解p的失效函数值，并给出按KMP算法进行匹配的过程

j	0	1	2	3	4	5	6
next[j]	-1	0	0	0	1	2	1

优化后的 nextVal